

Clase 246 — Supply chain security: SBOM y SLSA

Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: SLSA Framework, NTIA "Minimum Elements for a SBOM" y NIST SP 800-218 (SSDF) 🕒 Duración estimada: **120 min** · Nivel: **Avanzado**

Objetivo

Proteger la cadena de suministro de software: saber exactamente qué contienen tus artefactos (SBOM), garantizar la integridad y procedencia de cómo se construyeron (SLSA, atestaciones, firmas), y establecer confianza verificable de extremo a extremo. Tras ataques como SolarWinds, Codecov y las campañas de typosquatting en npm/PyPI, la cadena de suministro es una prioridad regulada (Orden Ejecutiva 14028 de EE. UU.). Usaremos **Syft**, **Trivy** y **cosign**.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** qué es un SBOM, sus formatos (CycloneDX, SPDX) y sus elementos mínimos.
2. **Generar** un SBOM de código y de imágenes con Syft/Trivy.
3. **Consumir** un SBOM para responder "¿estoy afectado por este CVE?" en minutos.
4. **Describir** los niveles de SLSA y qué garantiza cada uno.
5. **Crear y verificar** atestaciones de procedencia firmadas con cosign.

Temas

#	Tema	Por qué importa
1	Ataques a la cadena de suministro	SolarWinds, Codecov, typosquatting: el nuevo frente
2	SBOM: qué es y para qué	Inventario verificable de componentes
3	CycloneDX vs SPDX	Los dos formatos estándar
4	Generar y consumir SBOM	Inventario + respuesta rápida a CVE
5	SLSA: niveles 1–4	Garantías crecientes de integridad de build
6	Procedencia y atestaciones	Cómo se construyó, firmado y verificable
7	Firma y verificación (cosign)	Confianza criptográfica del artefacto

Definiciones y características

- **SBOM (Software Bill of Materials):** inventario formal de componentes de un artefacto. *Característica:* permite responder al instante qué contiene cuando sale un CVE (ej. Log4Shell).
- **CycloneDX / SPDX:** formatos estándar de SBOM. *Característica:* CycloneDX orientado a seguridad; SPDX orientado a licencias/ISO 5962.

- **Procedencia (provenance):** metadatos verificables de cómo, dónde y con qué se construyó un artefacto. *Característica:* base de SLSA; responde "¿este binario salió realmente de este código y pipeline?".
- **SLSA:** Supply-chain Levels for Software Artifacts, marco de niveles de integridad de build. *Característica:* niveles crecientes (build reproducible, aislado, no falsificable).
- **Atestación:** afirmación firmada sobre un artefacto (SBOM, provenance, resultados de escaneo). *Característica:* verificable criptográficamente y almacenable junto a la imagen.
- **Orden Ejecutiva 14028:** normativa de EE. UU. que exige SBOM a proveedores. *Característica:* convierte el SBOM en requisito, no en opcional.

Herramientas y preparación

- **Syft** (Anchore) — genera SBOM de código e imágenes.
- **Grype / Trivy** — consumen SBOM para detectar CVE.
- **cosign** — firma imágenes y adjunta atestaciones (SBOM, provenance).
- **SLSA GitHub Generator** — genera provenance SLSA en Actions.

```
# Generar SBOM de una imagen en CycloneDX:
syft miapp:1.0 -o cyclonedx-json > sbom.cdx.json

# Escanear consumiendo el SBOM:
grype sbom:sbom.cdx.json
```

Laboratorio guiado

1. **Genera el SBOM** de un proyecto y de su imagen:

```
syft dir:./mi-proyecto -o spdx-json > sbom.spdx.json
syft miapp:1.0 -o cyclonedx-json > sbom.cdx.json
```

Inspecciona: componentes, versiones, licencias, hashes. 2. **Consume el SBOM para responder a un CVE.** Simula que sale un CVE en una librería: busca en el SBOM si está y en qué versión. Luego escanea con Grype/Trivy usando el SBOM como entrada. 3. **Adjunta el SBOM como atestación firmada** a la imagen:

```
cosign attest --predicate sbom.cdx.json --type cyclonedx \
  miregistry/miapp@sha256:<digest>
cosign verify-attestation --type cyclonedx miregistry/miapp@sha256:<digest> \
  --certificate-identity-regexp '.*' --certificate-oidc-issuer-regexp '.*'
```

4. **Genera provenance SLSA en el pipeline.** Configura el `slsa-github-generator` para que el workflow emita una atestación de procedencia del artefacto build.
5. **Verifica la procedencia.** Comprueba que el artefacto fue construido por tu pipeline y a partir del commit esperado (`cosign verify-attestation --type slsaprovenance`).
6. **Autoevalúa tu nivel SLSA.** Contrasta tu pipeline con los requisitos: ¿build con script versionado? ¿generación de provenance? ¿build aislado y no falsificable? Determina tu nivel actual y el siguiente objetivo.
7. **Publica el SBOM.** Adjúntalo al release para que tus consumidores puedan auditarlo.

Nota ética: la seguridad de la cadena de suministro es defensiva. No publiques SBOM con datos internos sensibles sin revisarlos; contienen el mapa de tus componentes.

Ejercicios

1. Genera SBOM del mismo artefacto en CycloneDX y SPDX y compara su estructura.
2. Usa un SBOM para determinar en 1 minuto si estás afectado por un CVE dado.
3. Firma y verifica un SBOM como atestación con cosign.
4. Configura la generación de provenance SLSA en un pipeline.
5. Verifica que un artefacto proviene del commit y pipeline esperados.
6. Autoevalúa el nivel SLSA de tu pipeline y define un plan para subir uno.

Reto verificable

Entrega un artefacto con SBOM y procedencia verificables de extremo a extremo.

Criterio de aceptación: (a) el pipeline genera un SBOM (CycloneDX o SPDX) del artefacto y lo publica; (b) el SBOM se adjunta como atestación firmada con cosign y la firma verifica; (c) se genera provenance SLSA verificable que ata el artefacto a su código fuente y build; y (d) se documenta el nivel SLSA alcanzado con evidencia.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El SBOM no lista dependencias transitivas	Se generó de manifests, no del artefacto construido. Genera el SBOM de la imagen/binario final.
"Tengo SBOM pero no me sirve"	No lo consumes. Intégralo con Grype/Trivy y en el proceso de respuesta a CVE.
<code>cosign verify-attestation</code> falla	Tipo o identidad incorrectos. Ajusta <code>--type</code> y los filtros de certificado.
Provenance dice que lo construyó una máquina desconocida	Build no aislado o token comprometido. Endurece el pipeline (clase 242) y usa runners efímeros.
SBOM desactualizado respecto al artefacto	Se generó fuera del build. Genera SBOM y firma en el mismo job que produce el artefacto.

Preguntas frecuentes

? ¿SBOM y SLSA compiten? No. El SBOM dice *qué* contiene el artefacto; SLSA garantiza *cómo* se construyó con integridad. Juntos dan visibilidad y confianza de la cadena.

? ¿CycloneDX o SPDX? CycloneDX está más orientado a seguridad (integra VEX, se usa mucho con herramientas de escaneo); SPDX es estándar ISO y fuerte en cumplimiento de licencias. Muchos generan ambos.

? **¿Qué nivel de SLSA necesito?** Empieza por generar provenance (nivel bajo) y sube hacia builds aislados y no falsificables según tu criticidad. No todos necesitan el nivel máximo.

? **¿El SBOM me protege de un ataque de supply chain?** No lo previene por sí solo, pero te permite responder en minutos ("¿estamos afectados por X?") y es la base para verificar procedencia y detectar componentes maliciosos.

Referencias

- SLSA — <https://slsa.dev/>
- NTIA Minimum Elements for a SBOM — <https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>
- CycloneDX — <https://cyclonedx.org/>
- SPDX — <https://spdx.dev/>
- Syft & Gype (Anchore) — <https://github.com/anchore/syft>

Siguiente clase

Clase 247 - Seguridad de APIs en el ciclo de desarrollo